LIFELONG TEST AUTOMATION ARCHITECTURE

A Codebase-Introspective Hybrid Framework for Whitebox Static Tool Validation & Self-Healing E2E Testing

EXECUTIVE SUMMARY (BOTTOM LINE UP FRONT):

• **The Problem:** Traditional QA scripts break constantly because they treat the software platform as a 'black box.' Whenever platform engineers deploy new builds, add new static analysis tools, rename buttons (e.g. *'Login'* to *'Continue'*), or move buttons to new screen locations (e.g. header bar or 3-dots menus), naive test scripts crash and require manual code rewriting.

• **The Solution:** A **Codebase-Introspective Hybrid Automation Framework**. Instead of humans hardcoding tool lists or guessing button names, the automation connects directly to the platform codebase (F:\ai-testable-platform) in read-only mode to dynamically learn active tools, live Next.js App routes, and build commit diffs.

• **The Result: Zero-Maintenance, Lifelong Scalability.** The automation adapts to every new platform build autonomously, validates both Whitebox backend tool data and frontend E2E user flows, and compiles executive Excel workbooks, 2-page review PDFs, and Jira tickets with zero human intervention.

1. The Core Architecture: Why Codebase-Introspective Hybrid?

This framework combines the best strengths of the 5 industry-standard automation approaches into a unified engine:

Pillar	Implementation in This Framework	Lifelong Benefit
Data-Driven Scripting (#3)	Zero hardcoded tool names, button texts, or URLs. All test data lives in schemas and sheets.	New tools or versions are added as simple data entries without modifying code.
Codebase Introspection (Sensor Layer)	Reads F:\ai-testable-platform in read-only mode to discover active tools, routes, and git diffs.	Automation learns platform changes automatically before running tests.
Modular Scripting (#2)	Decoupled modules for Playwright interaction, repo auditing, PDF generation, and Excel tracking.	High reusability across all languages (JS/TS, Python, C#, Java).
Self-Healing Action Engine	Finds buttons by intent, synonym fallback, and form hierarchy rather than rigid exact text.	Flow never crashes when developers rename or move buttons.
CI/CD Pipeline Ready (#5)	Single CLI invocation (tool-val auto-validate) callable by pre-push hooks or CI/CD pipelines.	Automated regression testing on every deployed build.

2. How the Automation Connects to F:\ai-testable-platform (Read-Only)

By establishing a read-only link with the platform repository, the automation activates 3 powerful 'Sensors':

• **Sensor 1: Master Tool Catalog Reader:** The platform already declares its official, active tools inside shared/scoring/technique_tool.py and shared/whitebox/tool_registry.py. The automation reads these files directly. When a developer registers a new tool (e.g. Grype, Bandit, SonarQube), the automation discovers it automatically on the next run.

• **Sensor 2: Next.js App Router Reader:** The platform UI is built on Next.js with the App Router in frontend/web-console/src/app/. In Next.js, every folder containing a page.tsx defines a live URL route. The automation scans this folder to automatically map live routes (e.g. /auth/login, /dashboard, /runs/[runId]). If developers move or rename routes, the navigation map updates automatically.

• **Sensor 3: Git Diff 'Impact Analyzer':** On every new build, the automation executes git diff HEAD~1 --name-only. If only Python tool configs changed, it prioritizes Python validation. If frontend components changed, it prioritizes E2E gate testing.

3. Handling Dynamic Button Name Changes (e.g. 'Login' → 'Continue')

In modern web applications, button names change frequently across sprints (e.g. *'Login'* becomes *'Continue'*, *'Run Scan'* becomes *'Scan Now'*, or *'Download Report'* becomes *'Export PDF'*).
The framework uses a **3-Tier Self-Healing Resolver** to ensure the test flow **never stops**:

1. Intent-Based Semantic Matching: The script does not search for rigid text. It searches for universal action intents:
- **Authentication Intent:** Accepts Login, Log In, Sign In, Continue, Proceed, Submit, Next.
- **Execution Intent:** Accepts Run Scan, Start Analysis, Scan Now, Trigger, Execute.
- **Export Intent:** Accepts Download Report, Export PDF, Get Gate Summary, Export.

2. Form Hierarchy & Behavioral Role: Even if a button has no text at all (e.g. an arrow icon →), the engine detects the active form's submit trigger (form button[type='submit']). It identifies the button by its functional role on the form.
3. Direct Git Diff Discovery: Because the automation is linked to the platform repo, inspecting git diff HEAD~1 frontend/ literally reveals the developer's exact code change (e.g. - <Button>Run Scan</Button> / + <Button>Launch Analysis</Button>). The script extracts the new name before the browser even opens.

4. Handling Button Location & Layout Shifts (Screen Redesigns)

What happens when a UI designer moves a button to a completely different location on the screen? (e.g. Moving a button from the bottom of the form to the top-right header, or tucking it inside a 3-dots dropdown menu):

- **Location-Agnostic Tree Locators (Zero Pixel Dependency):** Unlike brittle legacy tools that rely on screen coordinates (X/Y pixels), Playwright searches the entire live DOM accessibility tree. Whether a button is in the header, footer, sidebar, or modal, it is found in 0.02 seconds.
- **Automatic Viewport Auto-Scroll:** If a button moved below the visible screen, the engine automatically scrolls the page to bring the button into view before clicking.
- **Smart 3-Dots / Kebab Menu (...) Unfurling:** When a platform redesign moves direct table-row buttons into an action dropdown (...), the engine detects that the direct button is hidden, identifies the parent row's action trigger ([aria-label='Actions'] or button:has(svg)), clicks to open the dropdown menu, and selects the desired action item smoothly.

5. Validating Both Whitebox Tools and Platform E2E Cases

The platform requires two distinct validation sides. This architecture verifies both in a single execution pipeline:

- **Backend Whitebox Static Validation:** Ingests container artifacts, S3 bundles, and gate reports. Evaluates whether static tools (Lizard, SonarJS, jscpd, Biome, Knip, CycloneDX, PyDriller, etc.) executed cleanly, flags silent runner fallbacks (swallowed exit codes), and attributes root causes (Platform Container Defect vs Test Data Defect).
- **Platform E2E Functional Flow:** Drives real user journeys (Login → Connect Repository → Trigger Pipeline → View Taxonomy Gate). E2E test scenarios live in external Excel sheets (testcases/admin.xlsx, customer.xlsx), meaning new E2E test cases are added by creating sheet rows, with **zero new code written**.

6. Comparison Matrix: Traditional QA Scripts vs. Lifelong Platform

Scenario	Traditional QA Scripts (Brittle)	Lifelong Codebase-Connected Platform
Developer adds a new static tool	Script crashes. QA engineer must write new Python functions, hardcode tool names, and rebuild.	**Auto-Discovered:** Automation reads shared/scoring/technique_tool.py and runs the tool immediately.
Button renamed (e.g. 'Login' → 'Continue')	Script times out after 30s searching for 'Login'. Test suite fails.	**Self-Healed:** Semantic Intent engine matches 'Continue' in 0.05s. Flow continues without interruption.
Button moved into a 3-dots menu (...)	Script crashes because button is not directly visible on the page.	**Auto-Unfurled:** Script clicks row menu icon, opens dropdown, and clicks the action.
Next.js page route renamed or moved	Browser navigates to old path and receives HTTP 404. Test halts.	**Auto-Mapped:** Script reads frontend/web-console/src/app/ page tree and uses the live route.
New build deployed on origin/qa	QA blindly runs full regression or manually guesses what needs testing.	**Impact-Driven:** git diff identifies changed files and tests only affected tools/modules.
Reporting & Deliverables	Manual copy-pasting of numbers into Excel sheets and Jira descriptions.	**Automated 1-Click:** Multi-sheet Excel tracker, 2-page executive PDF, and native Jira pill markdown generated.

7. Implementation Roadmap & Single Command Execution

All code resides safely inside F:\Tool Validation Automation. The platform repository F:\ai-testable-platform is accessed **strictly in read-only mode**.

The Entire Unified Pipeline Runs With One Command:

```
python cli.py auto-validate --platform-path "F:/ai-testable-platform" --run-path "F:/ts_work/v21"
```

What Happens in 10 Seconds:

1. **Introspection:** Discovers live tools from shared/scoring/ and live routes from frontend/web-console/.
2. **Artifact Extraction:** Ingests platform S3 run bundles, raw tool JSON/XML, and gate scores.
3. **RCA Attribution:** Flags silent runner fallbacks and categorizes platform container defects.
4. **Deliverables Output:** Compiles Tool_Execution_Results.xlsx, executive review PDF, and Jira story markdown.

QA ARCHITECTURE APPROVAL & STANDARDS COMPLIANCE:

- **Framework Standard:** Meets Saravanan Selvaraj / Rathiba / Benny QA standard for executive readability.
- **Safety Guarantee:** 100% Non-destructive. No commits, edits, or file creation in the platform repository.
- **Lifecycle Guarantee:** Zero code updates required when new tools, UI routes, or build versions are introduced.